



Qué es la Tecnología Java?

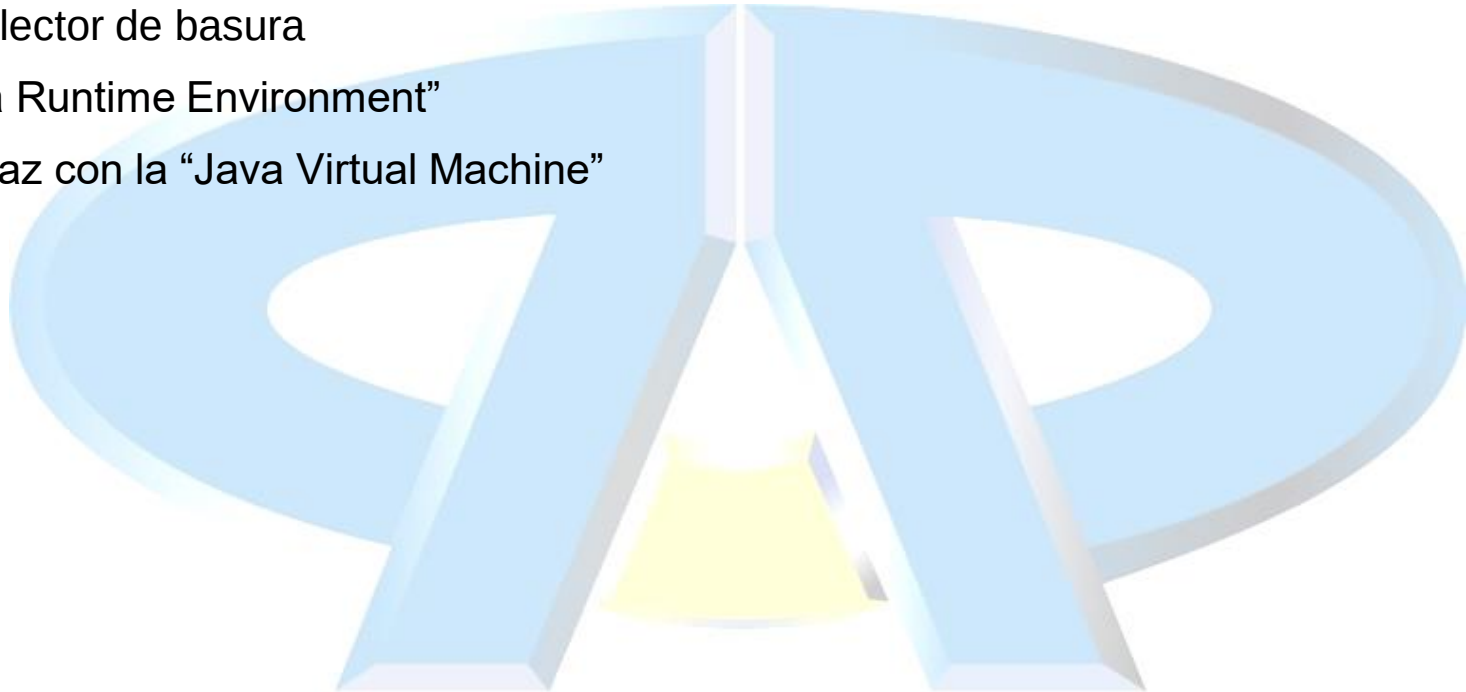
- Java es:
 - Un lenguaje de programación:
 - Sintaxis similar a C++
 - Permite el desarrollo de todo tipo de aplicaciones: convencionales o “stand-alone”, applets, aplicaciones Web, aplicaciones distribuidas, ...
 - Un entorno de desarrollo
 - Compilador: javac
 - Intérprete: java
 - Generador de documentación: javadoc
 - Empaquetador de ficheros: jar
 - Un entorno de despliegue y ejecución de aplicaciones:
 - Librerías básicas e intérprete
 - Java Runtime Environment (JRE) → Ejecución de aplicaciones “stand-alone”
 - JRE en el navegador → Applets
 - Contenedores de aplicaciones (e.g. Apache Tomcat) → Aplicaciones Web (Java EE, ahora Jakarta EE)

Características del lenguaje Java

- Facilidad de programación:
 - Elimina la aritmética de punteros y la gestión manual de memoria
 - Orientado a objetos
- Utiliza un entorno “interpretado”:
 - Reduce el tiempo del ciclo: Compilación – “linkado” – carga – prueba
 - **Portabilidad del código**: El mismo código compilado puede ejecutarse sobre diferentes plataformas sin necesidad de ser “recompilado”
- Facilita el desarrollo de aplicaciones “multi-thread”
- Permite la modificación dinámica de programas mientras éste se está ejecutando, mediante la descarga de módulos de código
- Valida los módulos de código antes de cargarlos

Elementos clave de la tecnología Java

- “Java Virtual Machine”
- Recolector de basura
- “Java Runtime Environment”
- Interfaz con la “Java Virtual Machine”



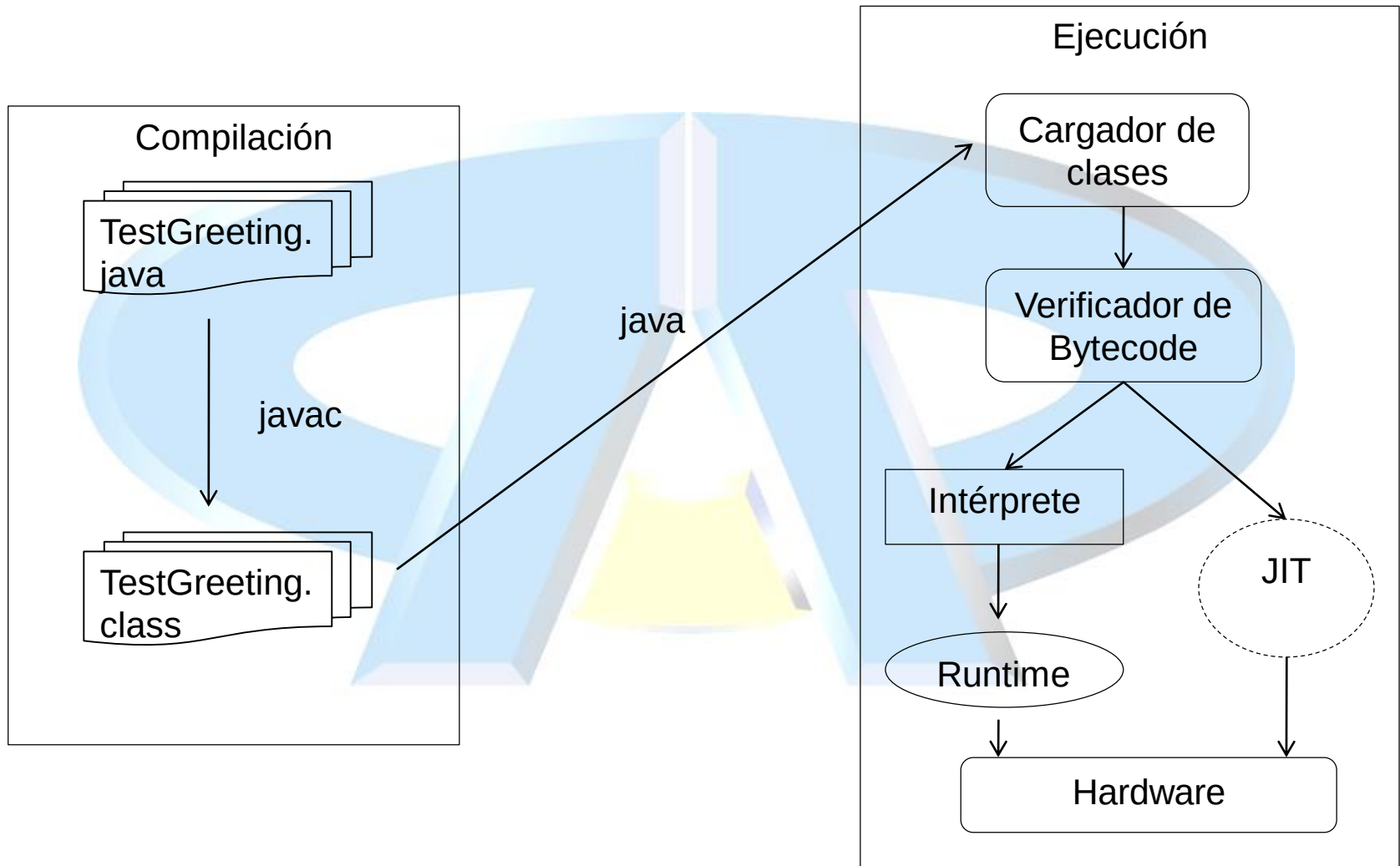
Java Virtual Machine

- Una máquina virtual (que no existe físicamente) que se implementa mediante la emulación en software de una máquina real. El código a ejecutar por la JVM se almacena en ficheros **.class**, cada uno de los cuales contiene, como mucho, una clase pública
 - Java Virtual Machine Specification
 - Bytecode
 - Intérprete compatible con Java
 - Entorno multiplataforma

Recolector de basura

- Gestión dinámica de memoria en tiempo de ejecución
- Utilización de punteros
- Liberación de la memoria reservada dinámicamente
 - En muchos lenguajes es el programador el responsable de liberar la memoria reservada
 - JVM incorpora un “thread” de sistema que hace un seguimiento de la memoria utilizada, que comprueba la memoria utilizada y libera la que ya no es necesaria
 - Se ejecuta de forma continuada, como un “thread” más de la aplicación
 - Su comportamiento varía mucho, en función de la implementación de la JVM

Java Runtime Environment



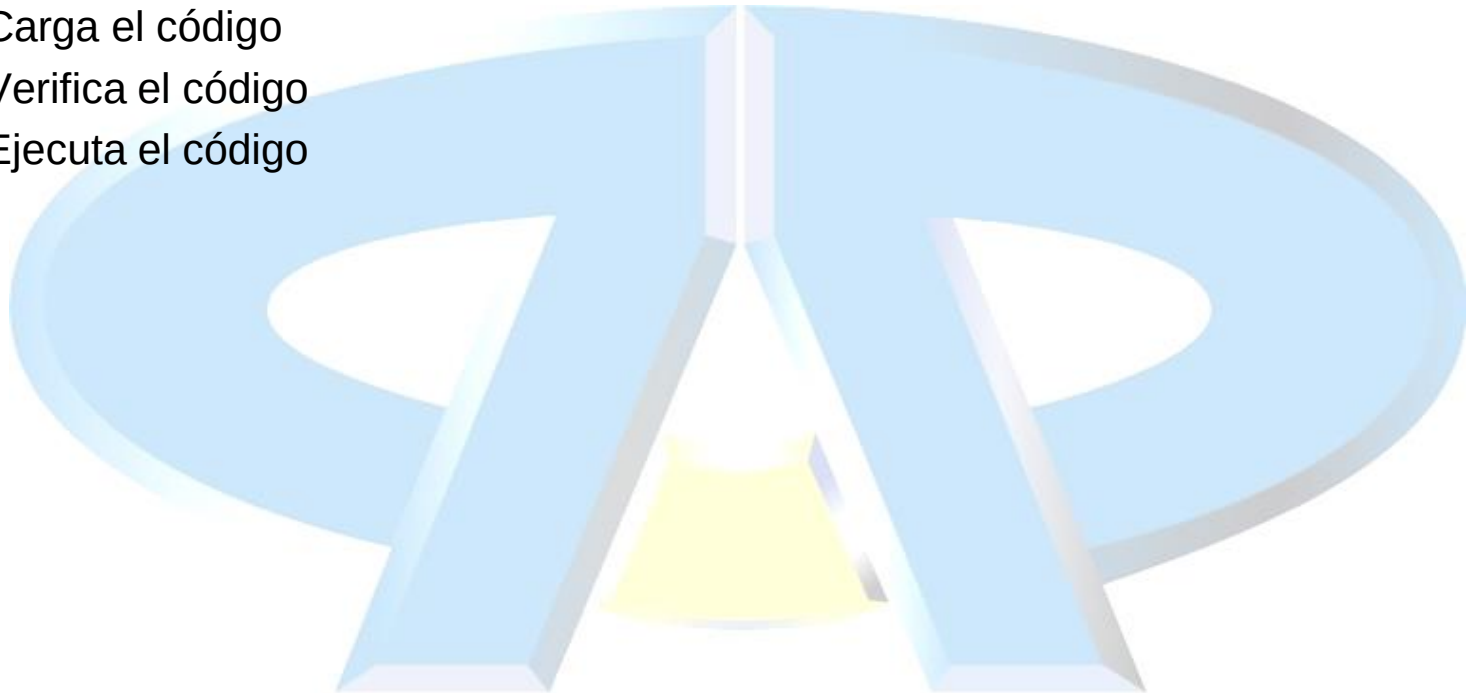
Java Runtime Environment

- El código fuente Java se compila para generar “bytecode”:
 - TestGreeting.java → TestGreeting.class
 - Compilador de Java: javac
- En tiempo de ejecución, el programa Java (en bytecode) se carga en memoria, se valida y es ejecutado por un intérprete (java)
- En el caso de los “applets”, el código es interpretado por la JVM del navegador (obsoletas)
- La JVM es la “máquina” que ejecuta el “bytecode”, haciendo las llamadas pertinentes al SO o al navegador:
 - Ventaja: Multiplataforma
 - Inconveniente: Cierta lentitud y pérdida de rendimiento (dependiente de la implementación de la JVM)
- JIT: Permite generar código máquina nativo para una determinada plataforma
- Desde la versión 10, se puede ejecutar código java sin compilarlo previamente

java MiClase.java

Tareas de la JVM

- Realiza tres actividades:
 - Carga el código
 - Verifica el código
 - Ejecuta el código



Cargador de Clases

- Carga todas las clases necesarias para la ejecución de un programa.
- Mantiene espacios de nombres separados para las clases del sistema de ficheros local y las clases cargadas a través de red
 - Ayuda a evitar la implementación de Caballos de Troya
- Define el orden de ejecución de las clases
- Utiliza referencias a memoria simbólicas → Se accede a la memoria real mediante referencias a una tabla
 - Dificulta los accesos ilegales a memoria

Verificador de bytecodes

- Ejecuta diferentes pruebas para asegurar que:
 - El código sigue las especificaciones de la JVM
 - El código no viola la integridad del sistema
 - El código no origina desbordamientos de pila
 - Los tipos de los argumentos son correctos en todo el código operacional
 - No hay conversiones ilegales

Aplicación sencilla en Java.

```
//  
////////////////////////////////////  
//  
// Ejemplo de aplicación "Hola Mundo"  
//  
public class TestGreeting {  
    public static void main (String[] args) {  
        Greeting hello = new Greeting();  
        hello.greet();  
    }  
}
```

Aplicación sencilla en Java.

```
//  
// Declaración de la clase Greeting.  
//  
public class Greeting {  
    public void greet() {  
        System.out.println("hi");  
    }  
}
```



Compilación y ejecución de TestGreeting.

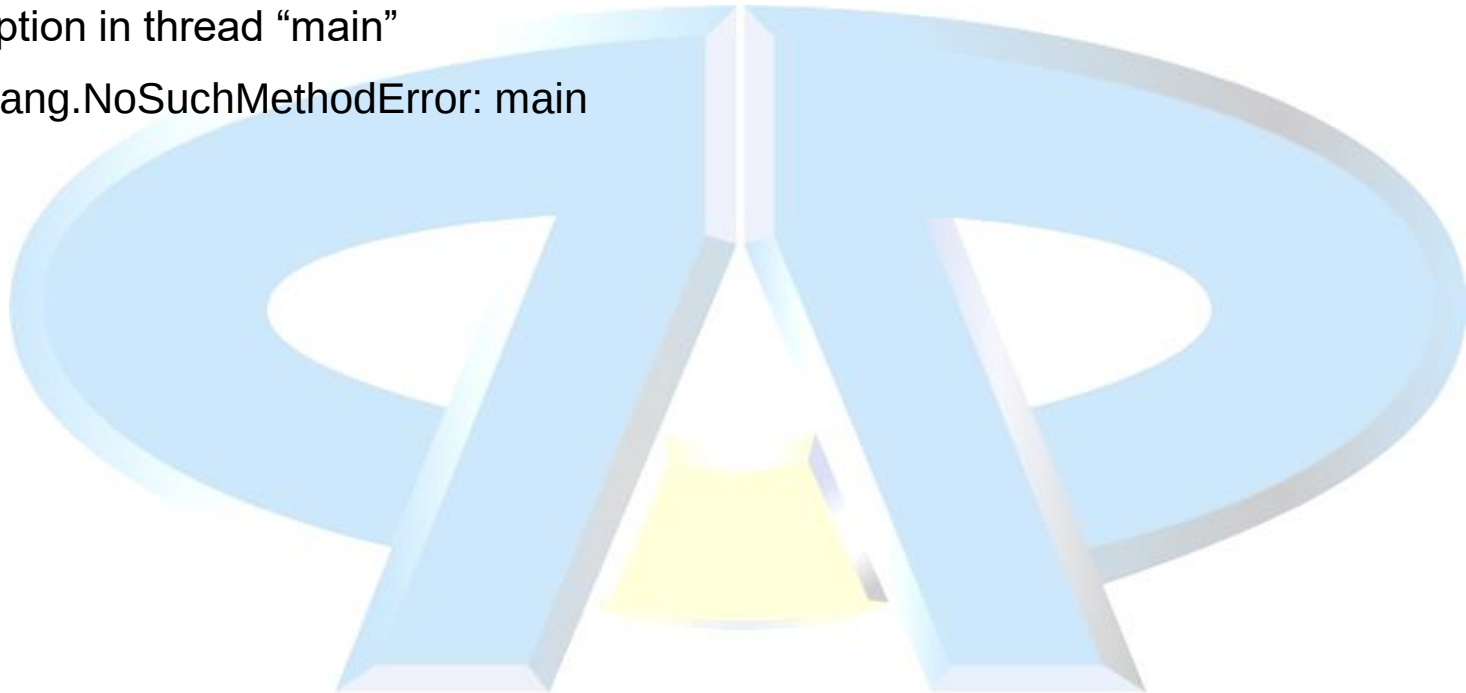
- Compilación de TestGreeting.java
 - `javac TestGreeting.java`
- Greeting.java se compila automáticamente.
- Ejecución de la aplicación
 - `java TestGreeting.java`
- Localización de errores comunes de compilación y de ejecución.
- Se debe configurar la variable de entorno `path = java_home/bin`.

Errores típicos de compilación.

- javac: command not found
- Greeting.java:10: Method printl(java.lang.String) not found in class java.io.PrintStream
System.out.printl(salutation + " " + whom);
- TestGreet.java:4: public class TestGreeting must be defined in a file called "TestGreeting.java".
- Solo una clase de alto nivel, no estática puede ser declarada pública en cada fichero fuente y debe tener el mismo nombre que el fichero fuente.

Errores en ejecución.

- Can't find class TestGreeting
- Exception in thread "main"
java.lang.NoSuchMethodError: main



JShell

- En la versión 9 de Java se ha añadido un interprete para Java, que permite probar comandos y obtener resultados inmediatos, sin tener que escribir clases completas
- Se ejecuta con el comando **jshell**

```
C:\Users\Rafa>jshell
| Welcome to JShell -- Version 15.0.1
| For an introduction type: /help intro

jshell> int a = 10;
a ==> 10

jshell> int b = a - 5;
b ==> 5

jshell>
```

Versiones y licencias Java

- Desde que Oracle adquirió Sun y tomó control de Java, hay una gran problemática sobre versiones y licencias
- En términos de código fuente, solamente existe uno, residente en el proyecto OpenJDK (<http://openjdk.java.net/projects/jdk/>)
 - No es una compilación distribuable
 - Los proveedores crean las compilaciones, las certifican y las distribuyen
- Oracle proporciona dos distribuciones:
 - Compilación OpenJDK - <http://jdk.java.net/>
 - Gratuita y sin marca, no hay actualizaciones cuando hay una nueva versión
 - OracleJDK - <https://www.oracle.com/java/technologies/downloads/>
 - Compilación comercial de marca con licencia y soporte
- Existen otras distribuciones de proveedores alternativos:
 - AdoptOpenJDK <https://adoptopenjdk.net/>
 - Amazon Corretto <https://aws.amazon.com/corretto/>
 - Azul <https://www.azul.com/products/zulu-enterprise/>
 - ...

Versiones y licencias Java

- Desde Java 9, las versiones de JDK aparecen cada 6 meses y una LTS (Long Term Support) cada 3 años

<https://openjdk.org/projects/jdk/>

Java 8 (LTS)	Mar 2014
Java 9	Sep 2017
Java 10	Mar 2018
Java 11 (LTS)	Sep 2018
Java 12	Mar 2019
Java 13	Sep 2019
Java 14	Mar 2020
Java 15	Sep 2020
Java 16	Mar 2021
Java 17 (LTS)	Sep 2021
Java 18	Mar 2022